AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

# Testing and debugging functions

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

## Freeze the expectation. Reveal the defect. Rerun the suite.

Design tests that distinguish a correct function from a plausible mistake and preserve repair evidence.

---

**Keep one habit:** Labelled suites → exact-boundary graphic → complete assertion program → constant-result defect → selected fresh boundary.

---

MAIN LESSON · SECTION 1

## Which test would distinguish the rule from a plausible mistake?

> Set expected results before running and preserve every mismatch.

### Your learning goals

I can design revealing tests for a function.

I can compare expected and actual results.

I can repair a defect and rerun earlier cases.

## Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from Reading structured data. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Return values and local state. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Design revealing tests for a function
- Compare expected and actual results
- Repair a defect and rerun earlier cases
- For capacity 9 and used 2, what should remaining return?

> **My starting explanation**

## Your end goal

Trace testing and debugging functions and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

**Your learning target**

**Remember before reading**

**Retrieve what you already know**

**This week’s end goal**

---

MAIN LESSON · SECTION 2

# A passing example can be a coincidence

> Test subtraction against independently calculated answers under the stated valid-input contract.

## Read the testing contract

A function can pass one example by coincidence. Start with a **written rule**, calculate expected results independently, and include ordinary, zero and **boundary** cases.

We will test a subtraction function under the contract 0 <= used <= capacity.

## Your first explanation

> **List the given facts and one fact that is not supplied.**

---

MAIN LESSON · SECTION 3

# Compare actual results with an independent expected column

> A printed comparison can continue; a failing assertion stops the run.

## Complete main subtraction test script

```
def remaining(capacity, used):
    return capacity - used

cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]
for capacity, used, expected in cases:
    actual = remaining(capacity, used)
    print(actual == expected)
```

## Match each input to its expected result

### Follow the evidence

| STEP OR CASE | EVIDENCE | MEANING |
| --- | --- | --- |
| Ordinary case | 8 and 3 | Expect 5 |
| Full case | 8 and 8 | Expect 0 |
| Unused case | 8 and 0 | Expect 8 |

Read each row from left to right and connect the evidence to the stated rule.

## Keep the practice suites and contracts labelled

| Separate complete suite | Expected subtraction outcomes |
|---|---|
| Main script · capacity 8 | (8,3)→5; (8,8)→0; (8,0)→8 |
| W1/W2 · capacity 6 | (6,2)→4; (6,6)→0; (6,0)→6 |
| Main assertion picture | tiny(x) means integer x<3; test 2→True,3→False,4→False |
| Selected assessment | Its own below_limit definition and 9/10/11 boundary; do not substitute the picture's 3 threshold |

## Read tuple unpacking, equality and assertion behavior

Each parenthesized group is a tuple, an ordered group of values. The loop unpacks each group into three names. Equality compares the actual result to the independently calculated expected result.

The worked script prints True three times. An assert actual == expected could instead stop at the first mismatch with **Assertion**Error.

## Explain the deciding check

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

**Connect the picture and the explanation**

# Compare an ordinary passing test with the revealing boundary

> Removing a failing check can conceal a defect without repairing it.

## Predict the supported change

- A faulty function always returns 4. Which of your three cases pass and fail?

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

## Predict the pictured variant

- Change: Remove the test at 3 and keep only 2 and 4.

### ▼ Check after predicting

**Prediction**: The buggy <= function passes those remaining tests.

Reason: Both procedures agree away from the equality **boundary**.

### ▼ Earlier wording, alternatives and picture transcripts

**Compare a familiar case with a changed case**

**Change one thing in the pictured case**

# Use the exact boundary to reveal the inclusive bug

> Keep the expected strict rule unchanged when the candidate is wrong.

## Read the complete tiny contract and test scope

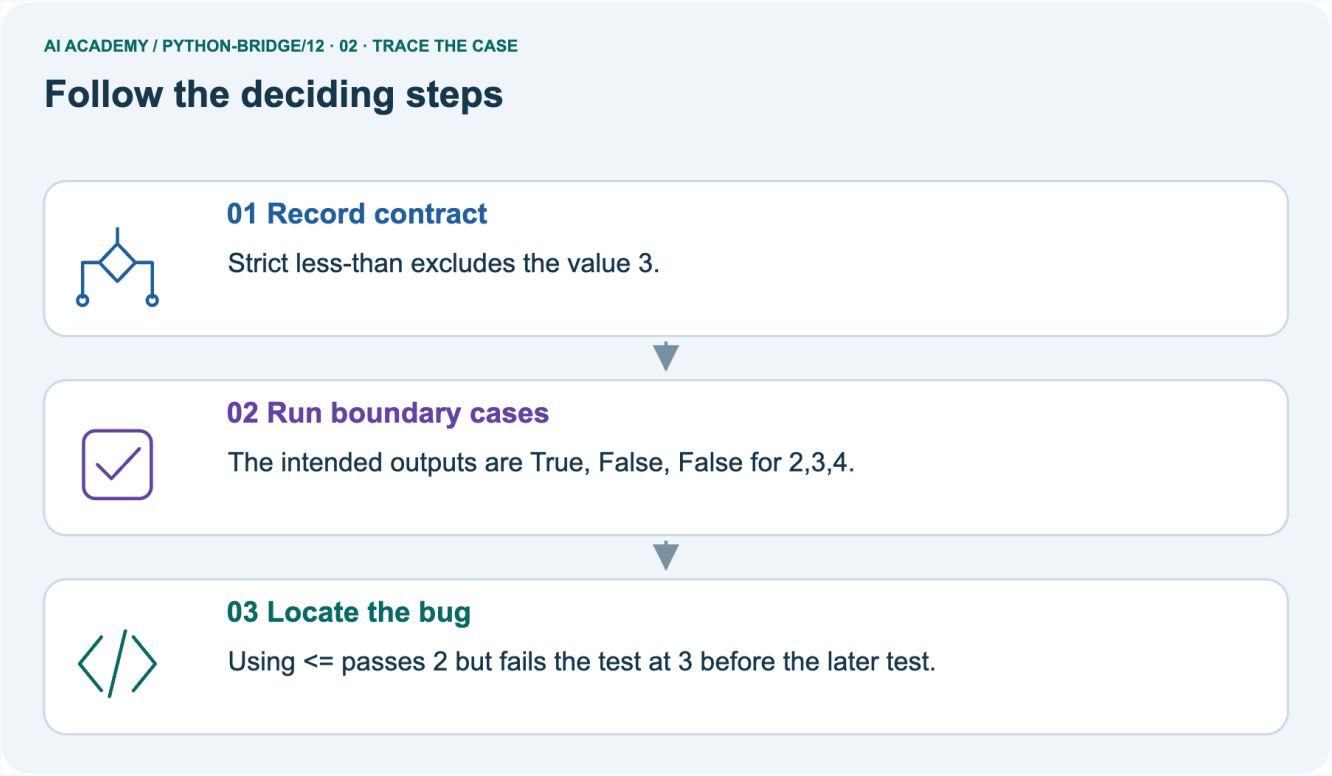
Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.

Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False). Candidate bug: return x<=3.

The complete tiny program below uses assertions in normal test execution. Correct <3 passes 2,3,4 and prints the completion line. Changing only < to <= passes the first assertion, fails at 3 with AssertionError and never runs the assertion for 4 or final print. The graphic compares expression results one at a time; it is not a trace claiming later assertions executed after failure.

The completion message is a literal string, not an automatic count of executed checks. If removing a test, relabel it for the two retained checks. These are local test checks, not a production input-validation mechanism.

## Trace the deciding boundary




Follow the labelled picture with the matching explanation. Open the vector diagram

### Read the picture one step at a time

| STEP | WHAT HAPPENS AND WHY |
| --- | --- |
| 1. Record contract | Strict less-than excludes the value 3. |
| 2. Run boundary cases | The intended outputs are True, False, False for 2,3,4. |
| 3. Locate the bug | Using <= passes 2 but fails the test at 3 before the later test. |

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

## Inspect where the two expressions differ

### THE EXACT BOUNDARY REVEALS THE BUG

Compare each expression separately for the same supplied integer.

| | x = 2 | x = 3 boundary | x = 4 |
|---|---|---|---|
| Intended: x < 3 | True | False | False |
| Bug: x <= 3 | True | True | False |

Only x = 3 distinguishes these versions. Tests at 2 and 4 both miss the bug.

A failing assertion stops its run; later assertions are not evaluated.

The table compares expressions, rather than claiming all checks ran after failure.

## Complete assertion suite

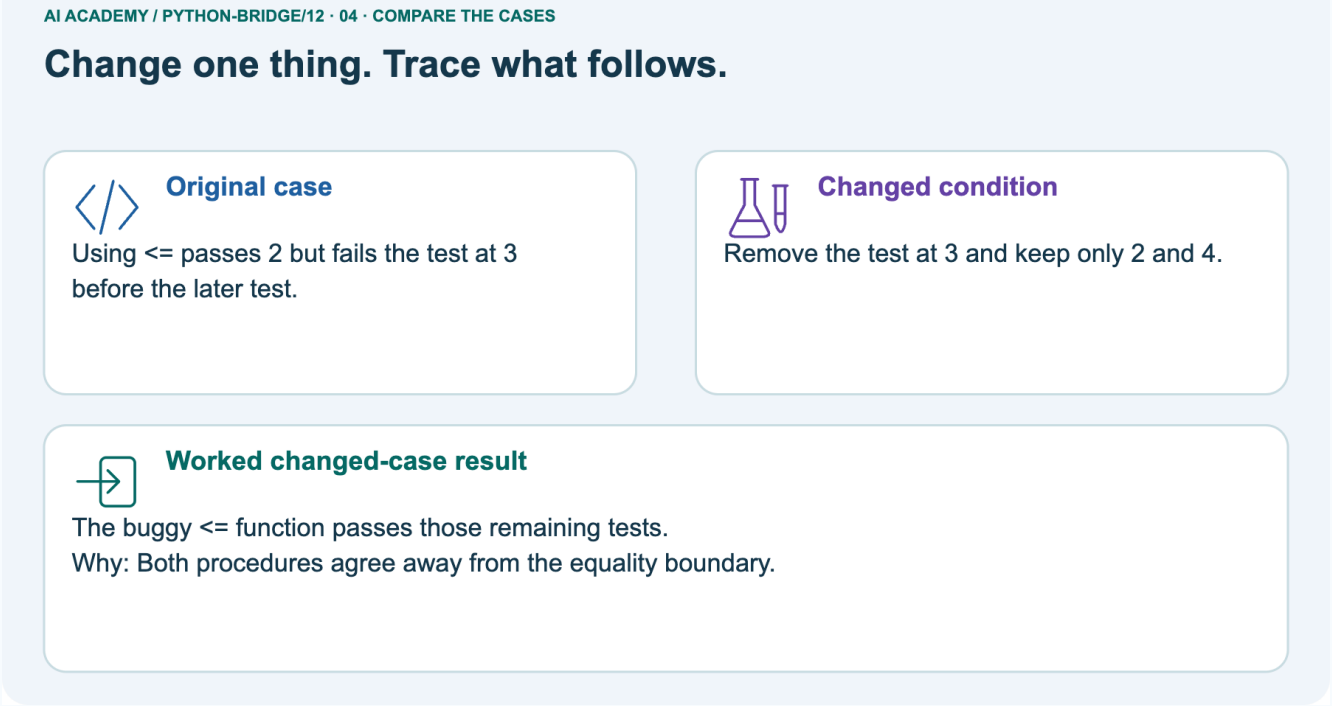
```
def tiny(x):
    return x < 3

assert tiny(2) is True
assert tiny(3) is False
assert tiny(4) is False
print("three checks passed")
```

## Predict removing the equality test

- Change one thing: Remove the test at 3 and keep only 2 and 4. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting




Follow the labelled picture with the matching explanation. Open the vector diagram

## State coverage and input-type limits

- Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

- Read the labelled picture: Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

▼ Earlier wording, alternatives and picture transcripts

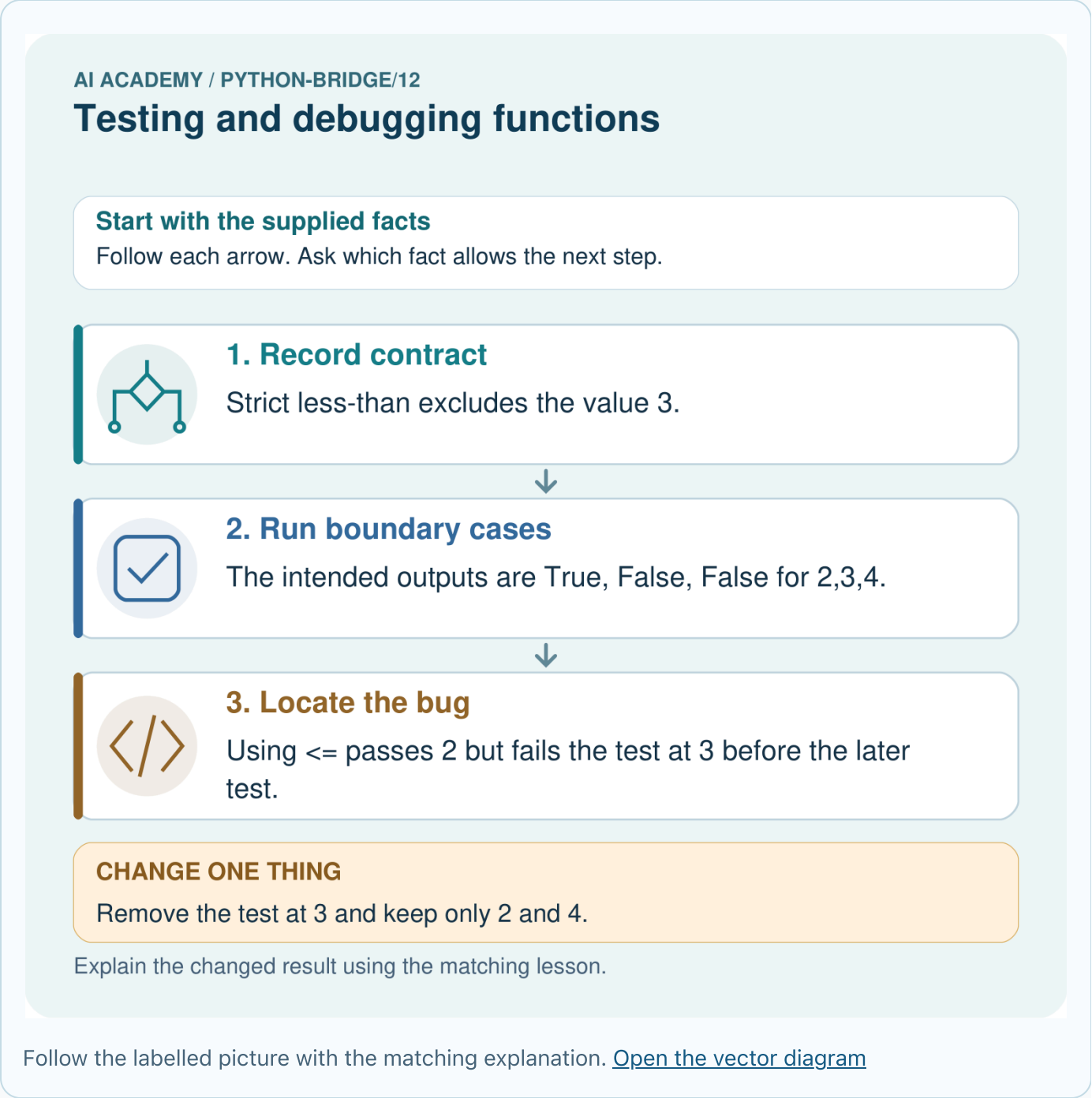
> **Would replacing the body with return 5 pass every worked case? Identify the first revealing test.**

### Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

## A pictorial worked case: Testing and debugging functions

### The facts we will use




Follow the labelled picture with the matching explanation. Open the vector diagram

### Why this result follows

### What this example does not establish

MAIN LESSON · SECTION 7

# Find the faulty operator without changing expected results

> A zero-used case can accidentally pass both addition and subtraction.

## Compare faulty addition against the main 8 suite

A faulty return capacity + used gives 11, 16 and 8. The unused case passes even though the rule is wrong. Repair the operator to subtraction and rerun all three cases.

Never change an expected result merely to match faulty **output**.

## Repair the requested 6/2 case and name a regression check

- A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a **regression** case.

**State one repair and the checks you would rerun.**

▼ Earlier wording, alternatives and picture transcripts

**Locate the earliest unsupported step**

MAIN LESSON · SECTION 8

# Practise the capacity 6 suite with less help

> The W2 constant 4 bug is tested on this suite, not the main capacity 8 suite.

## Read W1 expected values

- Give expected results for remaining(6, 2), remaining(6, 6) and remaining(6, 0).

> **Would replacing the body with return 5 pass every worked case? Identify the first revealing test.**

> **Explain your prediction to a partner before comparing results.**

## Complete W2 constant-result check

Use workbook W2 for the connected example “Testing and debugging functions”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

| Plan | Do | Check |
| --- | --- | --- |
| What is given? Which rule or procedure applies? | Record each intermediate step. | Does the result follow? What remains unknown? |

## Audit run-success-as-proof reasoning

- Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

## Explain the pictured boundary test

- Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

> **My explanation using a pictured step and the given facts**

## Reserve selected work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

### Practise with less help: Find a revealing case

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

### Check your reasoning

### Explain the picture

MAIN LESSON · SECTION 9

# Test six complete defect and coverage situations

> Use a candidate that can pass an easy case while still being wrong.

## All supplied candidates and outcomes to predict

| Complete test/variant | Expected evidence |
|---|---|
| A · Constant 4 candidate on(6,2),(6,6),(6,0) | Actual 4,4,4 versus expected 4,0,6; first passes by coincidence |
| B · Repair constant 4 to capacity-used | All three W2 cases match; preserve original failures |
| C · Faulty capacity+used on main 8 suite | Actual 11,16,8; only zero-used case passes |
| D · Tiny candidate x<=3 with full 2/3/4 assertions | First fails at 3; later assertion and completion print not reached |
| E · Same tiny bug, remove test 3 and keep 2/4 | Both retained assertions pass; defect remains |
| F · Allowed extra case (0,0), then proposed outside-contract inputs | Remaining 0 is expected for (0,0). For outside types/values, first define handling; existing cases cannot certify it. |

## Complete W2 candidate and cases

```
def remaining(capacity, used):
    return 4  # intentional candidate defect: ignores both arguments

cases = [(6, 2, 4), (6, 6, 0), (6, 0, 6)]
for capacity, used, expected in cases:
    actual = remaining(capacity, used)
    print(actual, expected, actual == expected)
```

## Predict, run or trace, repair and rerun

1. Predict A from the complete constant-four candidate and frozen capacity 6 suite, then run or paper trace and preserve actual/expected pairs. Repair only the body expression and rerun every earlier case for B. Compare the separate addition candidate C under the main 8 suite. For D/E, use the complete tiny assertion program; record the first failure and the checks never reached. Removing a test is not a repair.

   State the current valid-input contract before proposing F; do not invent a rejection behavior that was never specified. A test report should include code version, exact cases, expected results, actual/error record, repair and regression outcomes. Keep the independent below_limit 10 script unused until the first individual attempt.

## Keep the evidence record

A faulty function always returns 4. Which of your three cases pass and fail?

| Record | Your evidence |
| --- | --- |
| Given input and fixed rule | |
| Prediction before checking | |
| Actual result, or “not run” | |
| First mismatch and one explanation | |
| Boundary or missing input and expected response | |

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Reference · Original correct capacity 8 suite

```
def remaining(capacity, used):
    return capacity - used

cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]
for capacity, used, expected in cases:
    actual = remaining(capacity, used)
    print(actual == expected)
```

▼ Earlier wording, alternatives and picture transcripts

**Predict, investigate and preserve the evidence**

**Plan → try → check**

---

MAIN LESSON · SECTION 14

# Try alone: reveal the selected threshold bug

> The selected 9/10/11 cases have their own boundary, distinct from tiny's 2/3/4.

## Selected complete test task and scope

Predict the **output**. If the function mistakenly uses x<=10, identify the first failing **assertion** and whether the final print happens. State the repair and which earlier tests must be rerun.

Do these cases prove all **input** types are handled?

```
def below_limit(x):
    return x < 10
assert below_limit(9) is True
assert below_limit(10) is False
assert below_limit(11) is False
print("three checks passed")
```

Use the selected function and assertion order exactly, with normal test execution. The changed <= version is a separate candidate; preserve its first failed assertion and skipped statements. Keep the original expected results unchanged and rerun all earlier cases after repair. No invalid-type handling is implemented or proved.

## Your response and help record

**My independent explanation, trace and limits**

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

## Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Additional practice · Earlier stock-function tests**

Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

**My independent procedure, result and limitation**

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

**Independent case: Use boundary tests to reveal a function defect**

**Attempt this without hints**

**Use feedback to improve**

---

MAIN LESSON · SECTION 11

# Finish a Python-readiness portfolio that preserves a repair

> Passing a small suite supports those cases, not every possible input.

## This week’s contribution

Add a labelled testing and debugging functions **evidence** sheet to Python readiness portfolio. Show how you can design revealing tests for a function.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week’s record; a matching answer without a trace is insufficient.

## Project checkpoint and evidence

| Project evidence | Record |
|---|---|
| Component and version | |
| Inputs permitted at prediction time | |
| Trace and comparison with earlier work | |
| Failure, limit and human review | |

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

**Evidence for your project**

---

MAIN LESSON · SECTION 16

# Finish: name the contract, revealing test and regression result

> A correct diagnosis preserves the mismatch instead of editing it away.

## Explain without notes

- Close your notes. Explain this goal in your own words: Design revealing tests for a function

**My deciding step, evidence and one remaining question**

## Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

# Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

## Reference · The words used in this procedure

> Explain each term using a particular code line.

### Vocabulary and response

| Word | Meaning |
| --- | --- |
| Test case | Inputs and an expected result |
| Boundary | An edge of a permitted range |
| Regression | A previously working case that fails after a change |
| Assertion | A check that raises an error when its condition is false |

**Use two terms to explain a step in this week's example.**

▼ Extra challenge · Explain, apply and challenge

REFERENCE · SECTION 10

# Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

## Choose a challenge

- Give expected results for remaining(6, 2), remaining(6, 6) and remaining(6, 0).
- A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a **regression** case.
- All three stock tests pass. Does that prove correct handling of negative counts?

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Earlier wording, alternatives and picture transcripts

**Explain → apply → challenge**

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

# Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

## Self-check and explanation

- Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling. A test outside the contract needs a separately specified expected behavior.

- All three stock tests pass. Does that prove correct handling of negative counts?

**All three stock tests pass. Does that prove correct handling of negative counts?**

**My limited claim and a useful next check**

▼ **Extra practice · A complete trace**

# Extra practice · A complete trace

> Predict before opening the supplied trace.

## Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

**My complete trace of the worked case**

▼ **Check the complete trace**

### Follow the evidence

| STEP OR CASE | EVIDENCE | MEANING |
|---|---|---|
| Ordinary case | 8 and 3 | Expect 5 |
| Full case | 8 and 8 | Expect 0 |
| Unused case | 8 and 0 | Expect 8 |

Read each row from left to right and connect the evidence to the stated rule.

▼ Earlier wording, alternatives and picture transcripts

**Complete evidence record**

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

# Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

## Checklist and first prediction

| By the end, I can | How I will show it |
| --- | --- |
| Design revealing tests for a function | Explain the deciding step, show a trace or test, and state one limit. |
| Compare expected and actual results | Explain the deciding step, show a trace or test, and state one limit. |
| Repair a defect and rerun earlier cases | Explain the deciding step, show a trace or test, and state one limit. |

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

# Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

## First question and hint

- Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three.
- Which labelled step matches this detail? Using <= passes 2 but fails the test at 3 before the later test.
- 1. Locate the bug
- 2. Record contract
- 3. Run **boundary** cases
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

### Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ Check the first explanation

After your attempt, compare with the supported explanation: Locate the bug Using <= passes 2 but fails the test at 3 before the later test. A revealing test distinguishes a correct implementation from a plausible mistake.

Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

## Changed-case question and hint

- Changed-case practice: Remove the test at 3 and keep only 2 and 4. Which conclusion is supported by the supplied facts?
- 1. The buggy <= function passes those remaining tests.
- 2. This one example guarantees correct results for every future **input**.
- 3. We can decide the result without examining the changed **input** or the procedure.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

### Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ Check the changed explanation

After your attempt, compare with the supported explanation: The buggy <= function passes those remaining tests. Both procedures agree away from the equality **boundary**.

## Repair and keep the record

Review Reading structured data (python-bridge/11). Goal: **Process** a list of records consistently. Short practice: Circle the included rows, calculate the sum and mean, and state coverage.

Repeat after adding the fourth row with count four.. Return to this week and explain what you changed.

Review Return values and local state (python-bridge/9). Goal: Use a returned value in a later calculation. Short practice: Trace the two returned values and explain why first stays seven.

What happens if the caller tries to print result directly?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

### Repair a missing building block

### Predict, run and debug

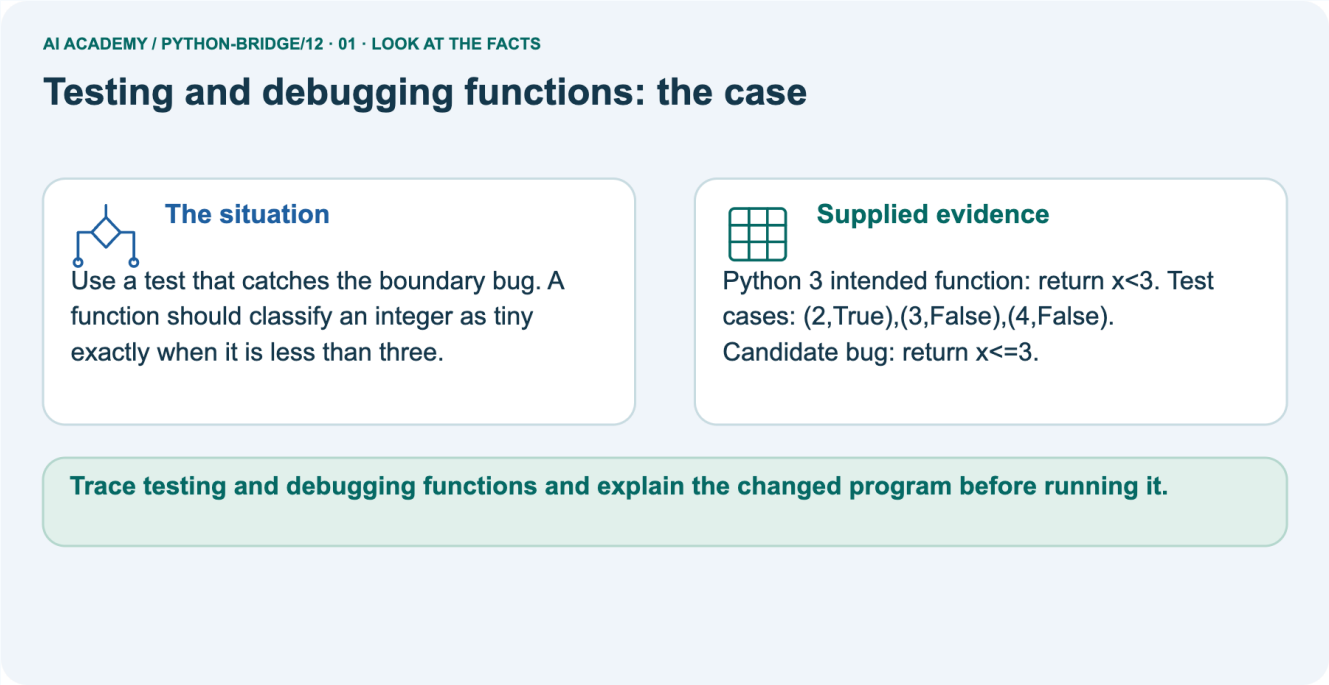
### Keep your completed work
</details>

REFERENCE · SECTION 18

# Reference · The six-picture journey

> Choose the picture needed for the present question.

## 01. Testing and debugging functions: the case



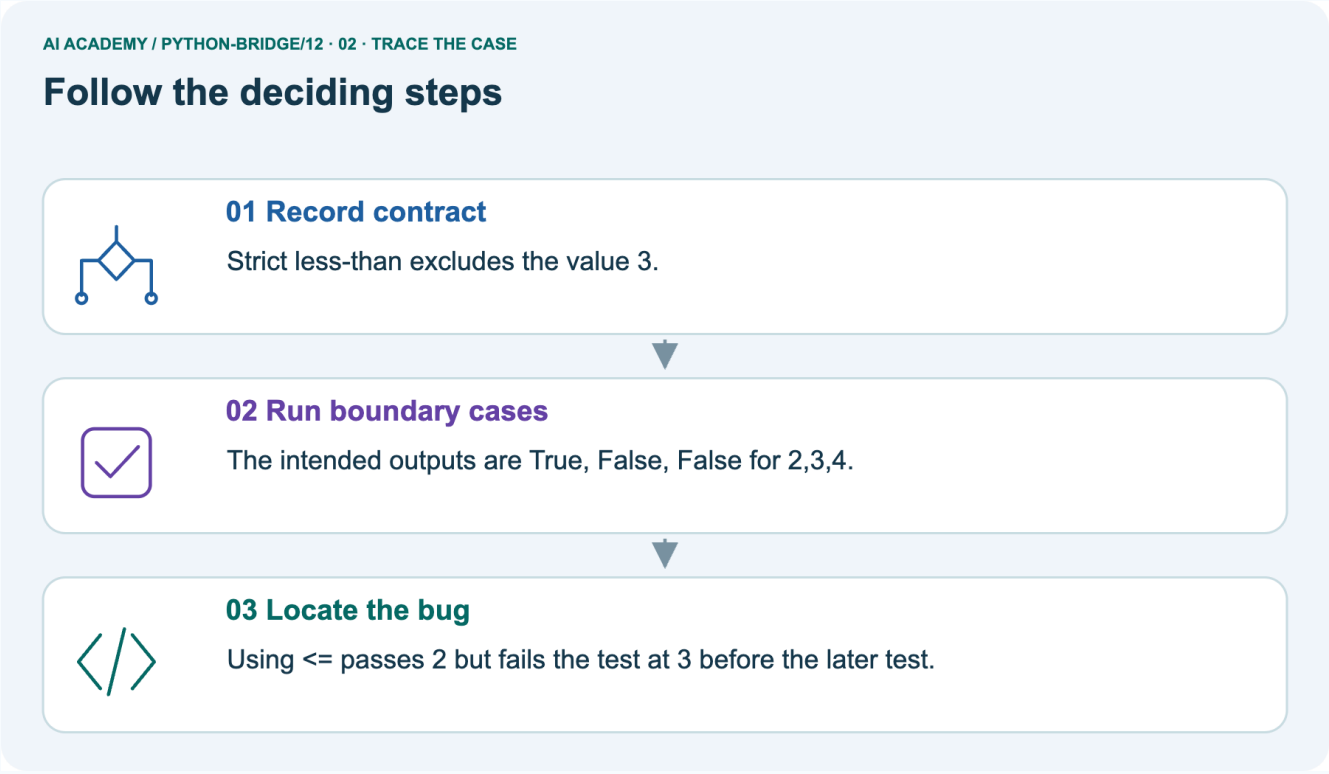

Follow the labelled picture with the matching explanation. Open the vector diagram

Use a test that catches the **boundary** bug. A function should classify an integer as tiny exactly when it is less than three. Supplied facts: Python 3 intended function: return x<3. **Test case**s: (2,True),(3,False),(4,False).

Candidate bug: return x<=3.

Point to the given fact that matters most. Explain why it matters.

## 02. Follow the deciding steps

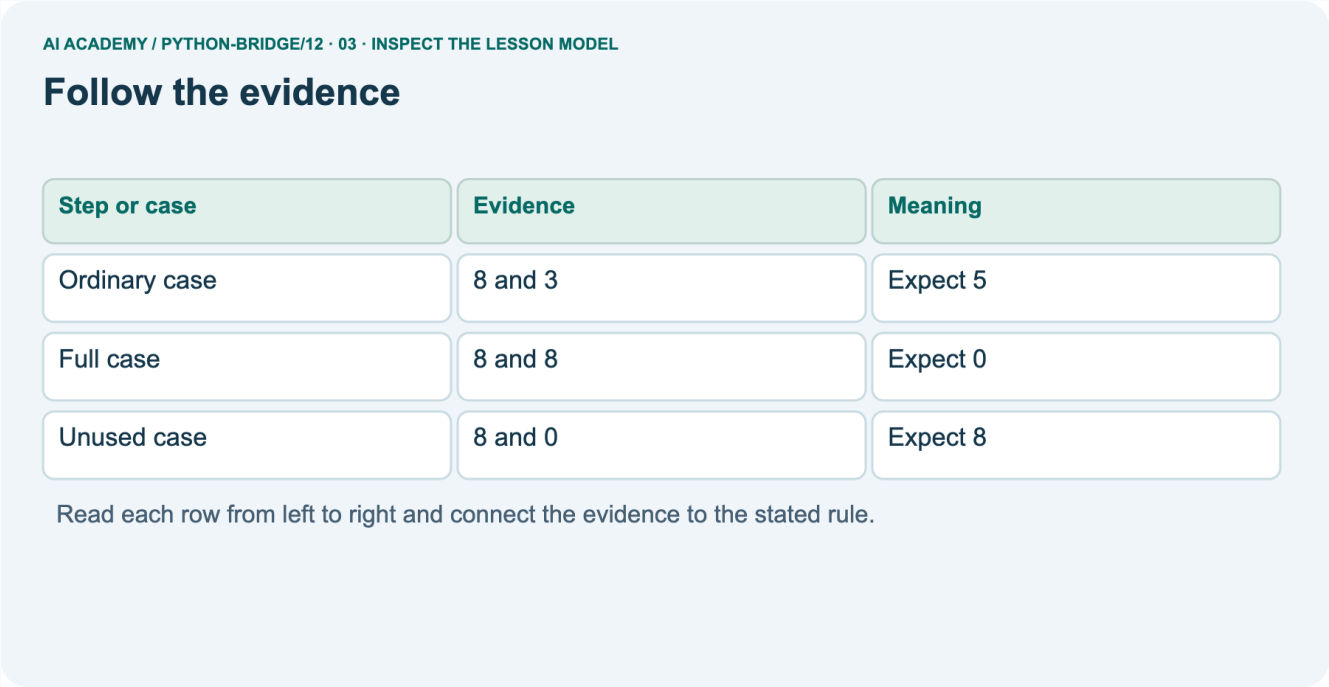



Follow the labelled picture with the matching explanation. Open the vector diagram

Record contract: Strict less-than excludes the value 3. → Run **boundary** cases: The intended **output**s are True, False, False for 2,3,4. → Locate the bug: Using <= passes 2 but fails the test at 3 before the later test.

Point to each numbered stage and explain which recorded **input** it uses.

## 03. Follow the evidence

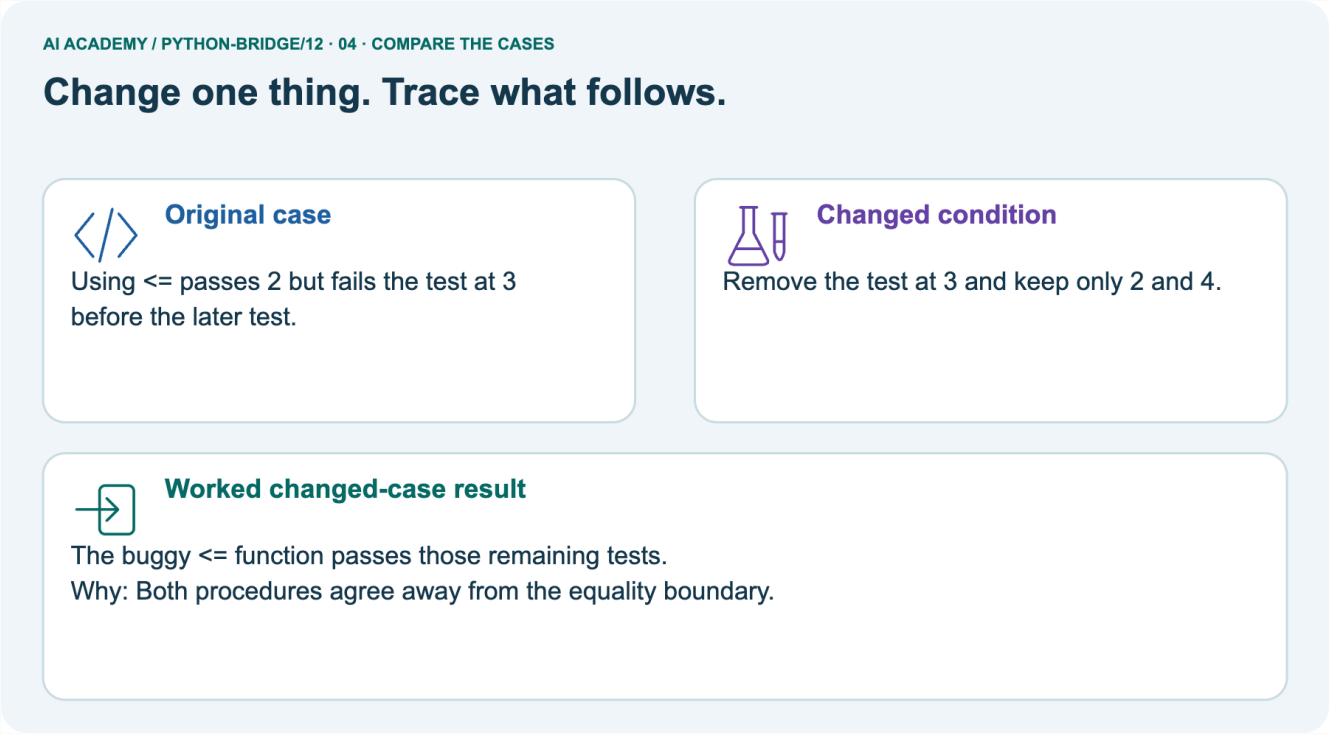



Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. Ordinary case | 8 and 3 | Expect 5. Full case | 8 and 8 | Expect 0. Unused case | 8 and 0 | Expect 8

Read the labels and values. Explain the relationship the model shows.

## 04. Change one thing. Trace what follows.



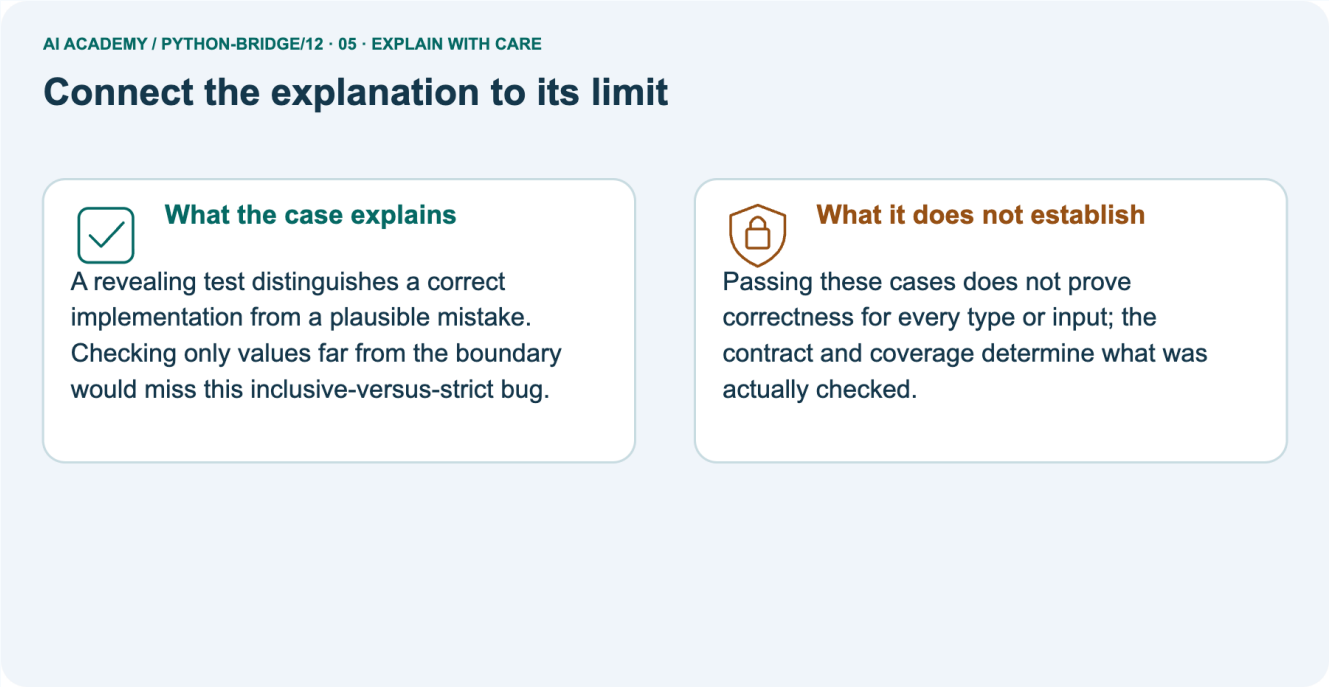

Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Using <= passes 2 but fails the test at 3 before the later test. Changed condition: Remove the test at 3 and keep only 2 and 4. Expected result: The buggy <= function passes those remaining tests.

Reason: Both procedures agree away from the equality **boundary**.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

## 05. Connect the explanation to its limit



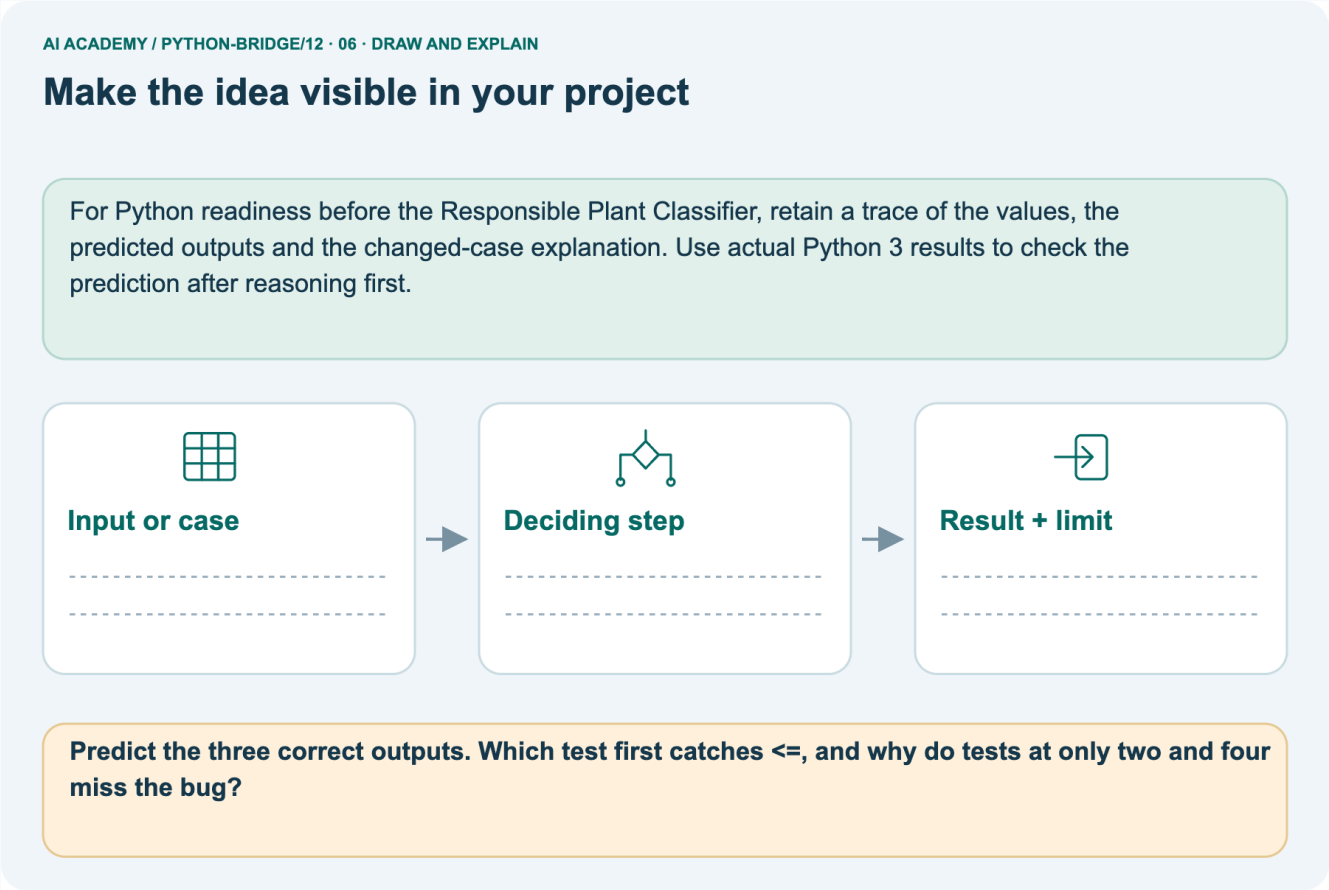

Follow the labelled picture with the matching explanation. Open the vector diagram

A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the **boundary** would miss this inclusive-versus-strict bug.

Limit: Passing these cases does not prove correctness for every type or **input**; the contract and coverage determine what was actually checked.

Explain the case, then point to a limit on the claim.

## 06. Make the idea visible in your project




Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Predict the three correct **output**s. Which test first catches <=, and why do tests at only two and four miss the bug?

Draw your project trace and justify the deciding step using **evidence**.

## ▼ Earlier wording, alternatives and picture transcripts

**01. Testing and debugging functions: the case**

**02. Follow the deciding steps**

**03. Follow the evidence**

**04. Change one thing. Trace what follows.**

**05. Connect the explanation to its limit**

**06. Make the idea visible in your project**

## ▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.